BASIC CAN COMMANDS

Arctos Robotic Arm — terminal quick reference

September 2026

Everything here is a command you can select from this PDF and paste directly into a terminal. Nothing needs editing except the CAN ID, and Section 3 gives the ready-made frame for every ID on this arm so you never have to work out a checksum by hand.

For why any of this works, see `Arctos_CAN_Interface.pdf`. For bringing up a specific joint from scratch, see the two SOP documents.

Contents

1 Set the interface up

The CANable presents as a serial device. `slcand` bridges it to a Linux network interface, and this has to be redone every time the adapter is plugged in or re-enumerates. Run these in order.

1.1 Check the adapter is there

```
lsusb
```

Expect a line containing `16d0:117e`. Then find the stable device name:

```
ls -l /dev/serial/by-id/
```

Use the `/dev/serial/by-id/` name rather than `/dev/ttyACM0`. The `ttyACM` number changes whenever the adapter re-enumerates; the by-id name does not.

1.2 Bring it up at 500 kbit/s

```
sudo pkill slcand
sudo ip link delete can0 2>/dev/null
sudo slcand -o -c -s6 /dev/serial/by-id/usb-Openlight_Labs_CANable2* can0
sudo ip link set can0 txqueuelen 1000
sudo ip link set up can0
```

Caution: The `-s6` flag is a coded index, not a number

`-s0` is 10k, `-s1` 20k, `-s2` 50k, `-s3` 100k, `-s4` 125k, `-s5` 250k, **`-s6` 500k**, `-s7` 800k, `-s8` 1M. This arm runs at 500 kbit/s so `-s6` is correct. Older scripts in this project used `-s3` and silently attached at 100 kbit/s, which looks like a dead bus.

1.3 Verify

```
ip -br link show can0
```

Expect `can0` followed by `UP`. That is the whole check.

Caution: Do not try to set or read the bitrate with `ip`

This does not work on this hardware and will fail:

```
sudo ip link set can0 up type can bitrate 500000
```

That netlink attribute is for native CAN controllers, not slcan. For the same reason `ip -details link show can0` reports `bitrate 0` and `clock 0` on a perfectly healthy adapter. That is normal, and it is not a fault worth chasing. The bitrate was fixed at attach time by `-s6`.

---

2 Shut it down so you can unplug

In this order. Stop any running control script first with Ctrl+C.

```
sudo ip link set can0 down
sudo pkill slcand
```

Then unplug the adapter. If a motor is energized and it is safe to release it, disable the coils first — see Section 3.

Caution: Dropping coils on a gravity-loaded joint lets it fall

Disabling a motor removes holding torque. On a bench motor that is harmless; on an assembled arm the joint may drop. Support it first, or leave it holding.

3 Ready-made frames for every joint

Every frame is `[command][data...][checksum]`, and the checksum includes the CAN ID:

```
checksum = (can_id + sum_of_all_other_bytes) & 0xFF
```

Because the ID is in the sum, **a frame copied from another joint is silently rejected**. The table below has the arithmetic already done. Find your joint's ID and use that row.

ID	Read enc.	Enable	Disable	E-stop	Enable state	Error
0x01	001#3132	001#F301F5	001#F300F4	001#F7F8	001#3A3B	001#393A
0x02	002#3133	002#F301F6	002#F300F5	002#F7F9	002#3A3C	002#393B
0x03	003#3134	003#F301F7	003#F300F6	003#F7FA	003#3A3D	003#393C
0x04	004#3135	004#F301F8	004#F300F7	004#F7FB	004#3A3E	004#393D
0x05	005#3136	005#F301F9	005#F300F8	005#F7FC	005#3A3F	005#393E
0x06	006#3137	006#F301FA	006#F300F9	006#F7FD	006#3A40	006#393F

3.1 Emergency stop

Work out your joint's e-stop line from the table and keep it pasted in a second terminal *before* you command any motion. For the joint most recently worked on, CAN ID `0x06`:

```
cansend can0 006#F7FD
```

Caution: Check the e-stop is addressed to the right joint

An earlier version of this project's documentation gave a joint's emergency stop as `cansend can0 001#F7F8` when the motor was not on ID `0x01`. It would have done nothing at all. Confirm the ID by scan, then take the matching row above.

3.2 Enable and disable coils

For CAN ID `0x06`:

```
cansend can0 006#F301FA
cansend can0 006#F300F9
```

The first enables, the second disables. An enabled joint holds position under power and can move; keep clear of the mechanism before enabling.

4 Watch what is on the bus

Everything:

```
candump can0
```

One joint only, by ID. This example filters to `0x06`:

```
candump can0,006:7FF
```

Log to a file while you work:

```
candump -L can0 > can_log.txt
```

Run `candump` in a second terminal while sending commands in the first. It is the fastest way to see whether a frame went out and whether anything answered.

5 Read-only queries

None of these move the motor or change its state, so they are safe to send at any time. Replace `006` and the checksum with your joint's row from Section 3.

What	Opcode	Expected at rest
Encoder position	`0x31`	stable, no jitter, no drift
Velocity	`0x32`	zero
Position error	`0x39`	small, tens of counts
Enable state	`0x3A`	`01` enabled, `00` disabled
Shaft protection	`0x3E`	`00`
Liveness	`0xF1`	answers `F1 01`

Read the encoder on `0x06`:

```
cansend can0 006#3137
```

The reply is `[0x31][6 bytes signed position][checksum]`, big-endian, on the 16384-counts-per-motor-revolution scale. To turn it into joint degrees:

```
joint_degrees = raw48 * 360.0 / 16384 / gear_ratio
```

6 Scan for what is actually on the bus

Never trust a config file or a table for a CAN ID. This sends only the read-only `0x31` query, never enables coils, and never commands motion:

```
cd ~/arctos_ws/src/arctos_can_control/arctos_can_control
python3 can_id_scan.py --channel can0 --ids 1-16
```

Exactly one responder is the healthy result. Two or more means another driver is powered on the bus with a colliding ID. None, with the board powered and wiring confirmed, is the signature of an RS485 board that has no CAN transceiver at all.

7 A safe test sequence

In order, from cold. Stop at the first step that does not do what it should.

1. Bring the interface up, confirm `can0 UP`.
2. Start `candump can0` in a second terminal.
3. Scan for the real CAN ID.
4. Read the encoder twice and confirm the value is stable.
5. Read the enable state and the shaft-protection state.
6. Paste your joint's e-stop line into a third terminal, ready but unsent.
7. Only now enable the coils, and only if the mechanism is clear.
8. Command one small move and confirm the encoder changed by what you asked.
9. Disable the coils.

8 When something is wrong

Symptom	**What to do**
`can0` does not exist	`slcand` died, usually because the adapter re-enumerated. Re-run the bring-up. Check `/dev/serial/by-id/` for the current name.
Adapter is plugged in but `can0` is missing	Expected. `slcand` does not restart itself after a replug.
`ip` shows `bitrate 0`	Normal on slcan. Not a fault.
Nothing answers any ID	Check panel CAN ID, CAN rate and CANRSP; check CAN_H, CAN_L and termination; confirm the driver has power. If still silent, check whether the board is the RS485 variant.
The command was accepted but nothing moved	The bus cannot tell "not driven" from "driven but cannot move". Disable the coils and turn the joint by hand.

Symptom	What to do
A move under-delivers, then delivers nothing	Stop. Do not resend — that only heats the motor. Back off and re-attempt once, or investigate the mechanism.
Motor is hot	Stop and let it cool. There is no temperature-read opcode, so nothing in software will warn you. Warm is normal; too hot to touch is not.